Data Visualization with Matplotlib & Seaborn

Prepared by: SO Vengleab · July 2026

Python for Data Visualization

Python provides a rich ecosystem of libraries designed to transform raw data into **meaningful insights**.

These tools allow for:

- Informative & customized plots
- Professional-grade aesthetics
- Simple and effective data storytelling

Matplotlib

The foundational library for static, animated, and interactive visualizations in Python.

Seaborn

Built on top of Matplotlib, it provides a high-level interface for drawing attractive statistical graphics.

Setup Data

Our Two Datasets

USED IN EVERY EXAMPLE TODAY

Load once, plot all lecture

```
In [1]:
data = pd.read_csv("car_sales.csv")
data.head()
```

```
In [2]:
loans = pd.read_csv("loans.csv")
loans[["loan_amnt", "term", "int_rate",
       "grade", "issue_d", "age"]].head()
```

	Make	Colour	Odometer (KM)	Doors	Price
0	Honda	White	35431.0	4.0	15323.0
1	BMW	Blue	192714.0	5.0	19943.0
2	Honda	White	84714.0	4.0	28343.0
3	Toyota	White	154365.0	4.0	13434.0
4	Nissan	Blue	181577.0	3.0	14043.0

	loan_amnt	term	int_rate	grade	issue_d	age
0	5000	36 months	13.56	C	Aug-18	33
1	4000	36 months	10.90	B	Mar-18	21
2	5000	36 months	23.40	E	Jul-18	56
3	5500	36 months	14.47	C	Dec-18	20
4	11200	60 months	16.14	C	Aug-18	23

Setup

00 · FOUNDATIONS

- pandas loads and reshapes the data — every plot starts from a DataFrame.
- matplotlib.pyplot (alias plt) is the drawing engine; fig, ax = plt.subplots() gives you a figure and its axes.
- seaborn (alias sns) sits on top of Matplotlib — every Seaborn chart is still a Matplotlib figure, so plt.show() displays both.

In [1]:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Bar Chart

SECTION 01

Compare a numeric total across categories — total sales price per car maker.

Bar Chart with Matplotlib

Why a Bar Chart?

BAR CHART · THE QUESTION

One row = one car sold

- Reading the raw transactions, the overall performance of each manufacturer is invisible.
- How many cars did each manufacturer sell, and how much revenue did each generate?
- Which manufacturer sold the most overall?

Insight: Group the rows by manufacturer, total the prices, and a bar chart answers all three at a glance.

In [4]:

```
data.head()
```

Out[4]:

	Make	Colour	Odometer (KM)	Doors	Price
0	Honda	White	35431.0	4.0	15323.0
1	BMW	Blue	192714.0	5.0	19943.0
2	Honda	White	84714.0	4.0	28343.0
3	Toyota	White	154365.0	4.0	13434.0
4	Nissan	Blue	181577.0	3.0	14043.0

Prepare the Data

BAR CHART · MATPLOTLIB

Original → expected format

- Matplotlib wants one number per bar.
- groupby("Make").sum() turns 1,000 rows into 4.

In [4]:

```
data.head()
```

Out[4]:

	Make	Colour	Odometer (KM)	Doors	Price
0	Honda	White	35431.0	4.0	15323.0
1	BMW	Blue	192714.0	5.0	19943.0
2	Honda	White	84714.0	4.0	28343.0
3	Toyota	White	154365.0	4.0	13434.0
4	Nissan	Blue	181577.0	3.0	14043.0

- **Insight:** Remember this groupby — in a few slides Seaborn does it for you automatically.

In [5]:

```
total = data[["Make", "Price"]].groupby("Make").sum()
total
```

Out[5]:

	Price
Make	
BMW	2501927.0
Honda	4107844.0
Nissan	2391288.0
Toyota	5563201.0

Bar · Basic Plot

BAR CHART · MATPLOTLIB

Plotting the aggregated table

- ax.bar(x, height) — categories on x, totals as bar heights.
- total.index holds the maker names, total["Price"] the sums.
- plt.show() renders the figure.

In [6]:

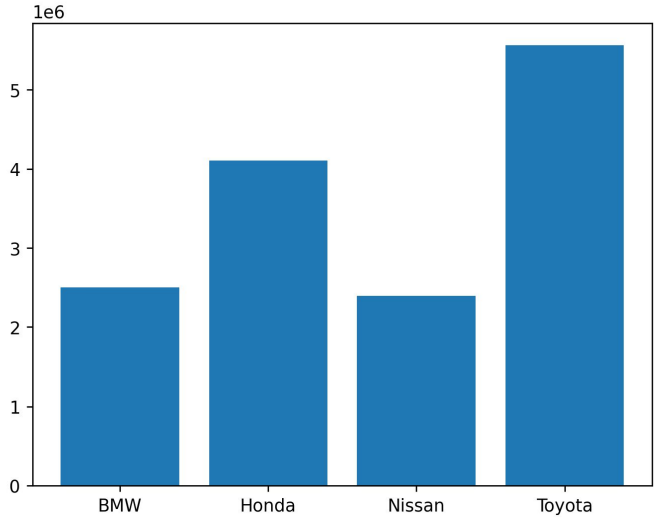
```
total
```

Out[6]:

Make	Price
BMW	2501927.0
Honda	4107844.0
Nissan	2391288.0
Toyota	5563201.0

In [7]:

```
fig, ax = plt.subplots()
ax.bar(total.index, total["Price"])
plt.show()
```



Bar · Colors & Title

BAR CHART · MATPLOTLIB

Styling is manual

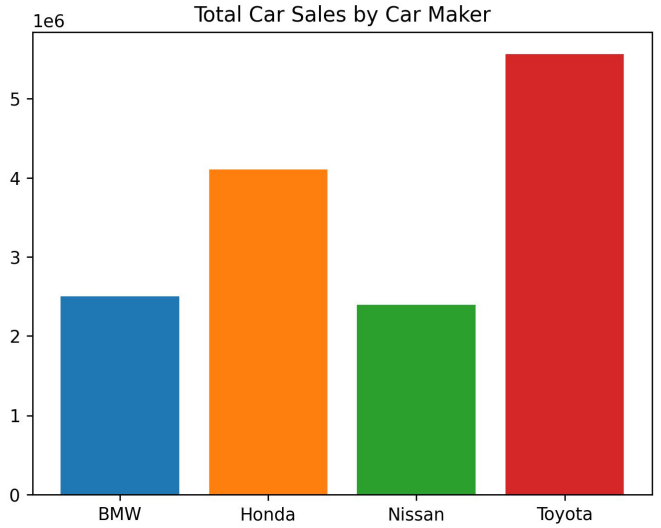
- color= takes one color per bar — you list them yourself.
- ax.set_title() adds the headline.
- Every styling choice in Matplotlib is an explicit argument.

In [6]:
```
total
```

Out[6]:

Make	Price
BMW	2501927.0
Honda	4107844.0
Nissan	2391288.0
Toyota	5563201.0

In [8]:
```
fig, ax = plt.subplots()
ax.bar(total.index, total["Price"],
       color=["tab:blue", "tab:orange",
              "tab:green", "tab:red"])
ax.set_title("Total Car Sales by Car Maker")
plt.show()
```



Bar Chart with Seaborn

Seaborn countplot

BAR CHART · SEABORN

No data prep at all

- sns.countplot takes the raw DataFrame — no groupby.
- It counts rows per category itself and labels the axes.
- The input on the left is the untouched raw table, not a summary.

In [9]:

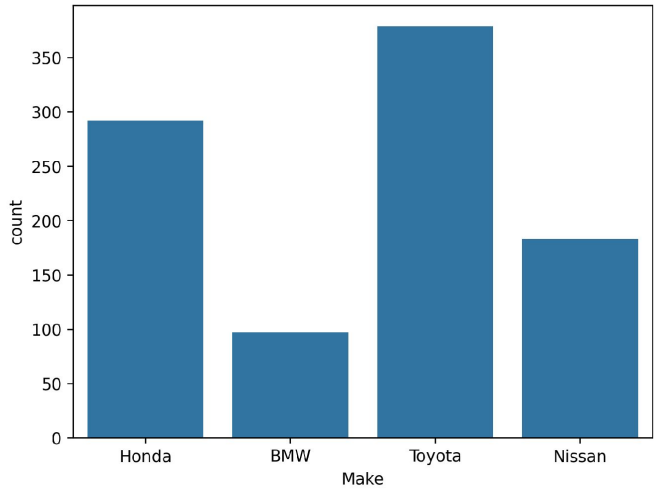
```
data[["Make", "Colour", "Price"]].head(3)
```

Out[9]:

	Make	Colour	Price
0	Honda	White	15323.0
1	BMW	Blue	19943.0
2	Honda	White	28343.0

In [10]:

```
sns.countplot(data=data, x="Make")
plt.show()
```



Seaborn barplot

BAR CHART · SEABORN

Aggregation built in

- estimator="sum" tells Seaborn to total Price per Make — the groupby from earlier, done internally.
- errorbar=None hides the uncertainty whiskers.
- Same chart as Matplotlib's, from the raw rows on the left.

In [9]:

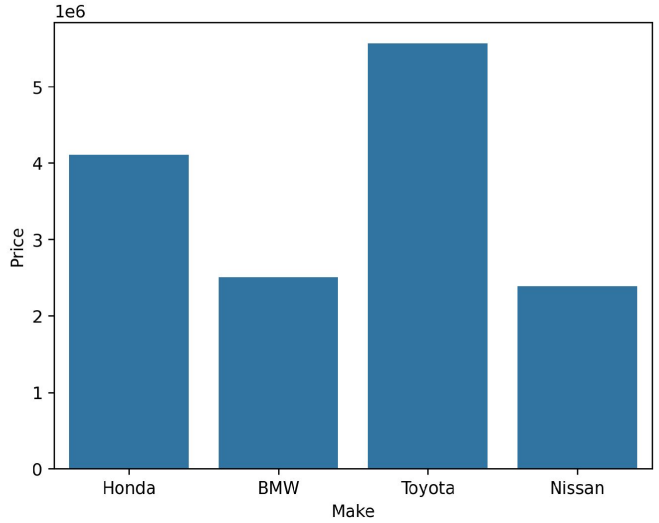
```
data[["Make", "Colour", "Price"]].head(3)
```

Out[9]:

	Make	Colour	Price
0	Honda	White	15323.0
1	BMW	Blue	19943.0
2	Honda	White	28343.0

In [11]:

```
sns.barplot(data=data, x="Make", y="Price",
            estimator="sum", errorbar=None)
plt.show()
```



Pie Chart

SECTION 02

Share of a whole — each maker's slice of total sales. (Matplotlib only: Seaborn has no pie chart.)

Why a Pie Chart?

PIE CHART · THE QUESTION

The four maker totals

- A bar chart compares sizes, but not each maker's share of the whole.
- What percentage of total revenue does each manufacturer hold?
- Does one manufacturer dominate, or is the market evenly split?

In [6]:
```
total
```

Out[6]:

Make	Price
BMW	2501927.0
Honda	4107844.0
Nissan	2391288.0
Toyota	5563201.0

- **Insight:** When the parts must add up to 100%, a pie chart shows the split better than bars.

Prepare the Data

PIE CHART · MATPLOTLIB

Original → expected format

- A pie needs one value per slice — total has it.
- Each total ÷ grand total = share of 100%.
- These are the numbers autopct will print.

In [6]:

```
total
```

Out[6]:

Make	Price
BMW	2501927.0
Honda	4107844.0
Nissan	2391288.0
Toyota	5563201.0

In [12]:

```
share = total["Price"] / total["Price"].sum() * 100
share.round(2).to_frame("Share (%)")
```

Out[12]:

Make	Share (%)
BMW	17.18
Honda	28.20
Nissan	16.42
Toyota	38.20

- **Insight:** Toyota at 38.20% — watch for that exact number on the next slide.

Pie · Basic Plot

PIE CHART · MATPLOTLIB

Same aggregated table

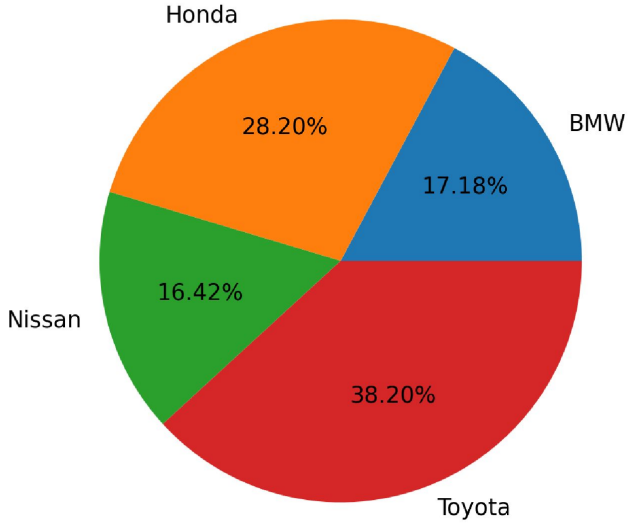
- ax.pie(values, labels=...) — one slice per row of total on the left.
- autopct="%.2f%%" prints the percentage on each slice.
- Seaborn has no pie chart — pies stay a Matplotlib job.

In [6]:
```
total
```

Out[6]:

Make	Price
BMW	2501927.0
Honda	4107844.0
Nissan	2391288.0
Toyota	5563201.0

In [13]:
```
fig, ax = plt.subplots()
ax.pie(total["Price"], labels=total.index,
       autopct="%.2f%%")
plt.show()
```



Pie · Styling

PIE CHART · MATPLOTLIB

Styling arguments

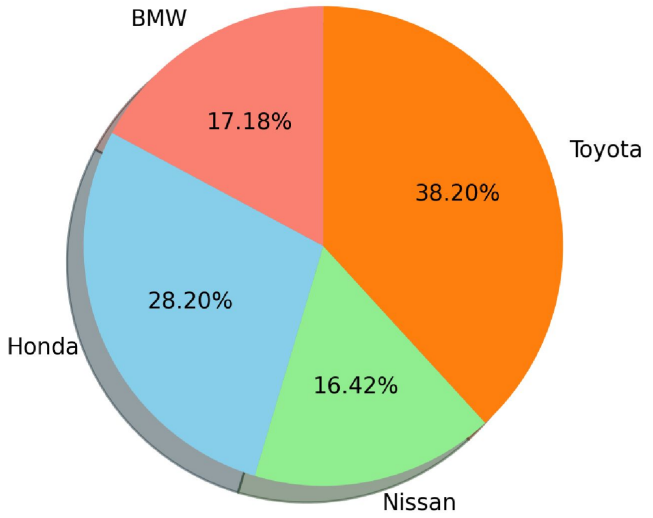
- colors= sets one color per slice, in data order.
- shadow=True adds depth under the pie.
- startangle=90 rotates where the first slice begins.

In [6]:
```
total
```

Out[6]:

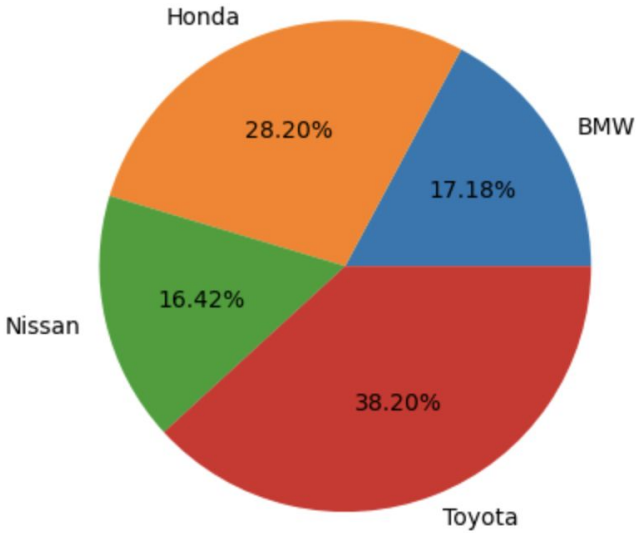
Make	Price
BMW	2501927.0
Honda	4107844.0
Nissan	2391288.0
Toyota	5563201.0

In [14]:
```
fig, ax = plt.subplots()
ax.pie(total["Price"], labels=total.index,
       autopct="%.2f%%", shadow=True,
       startangle=90,
       colors=["salmon", "skyblue",
               "lightgreen", "tab:orange"])
plt.show()
```



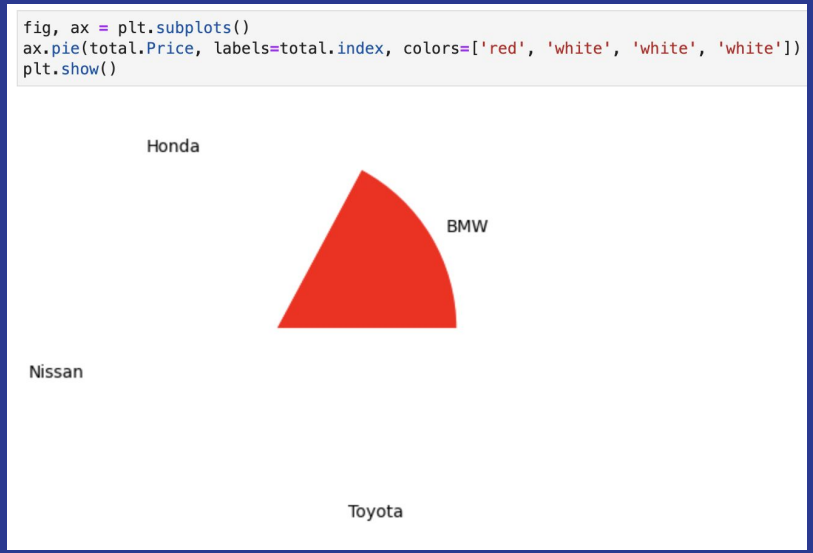
Auto Labeling

```
fig, ax = plt.subplots()
ax.pie(total.Price, labels=total.index, autopct='%.2f%%')
plt.show()
```



Coloring Slices

```
fig, ax = plt.subplots()
ax.pie(total.Price, labels=total.index, colors=['red', 'white', 'white', 'white'])
plt.show()
```



Pie · Explode & Hatch

PIE CHART · MATPLOTLIB

Highlighting one slice

- explode offsets slices — here only Toyota (0.1) pops out.
- hatch= fills each slice with a texture pattern, useful for black-and-white printing.

In [6]:

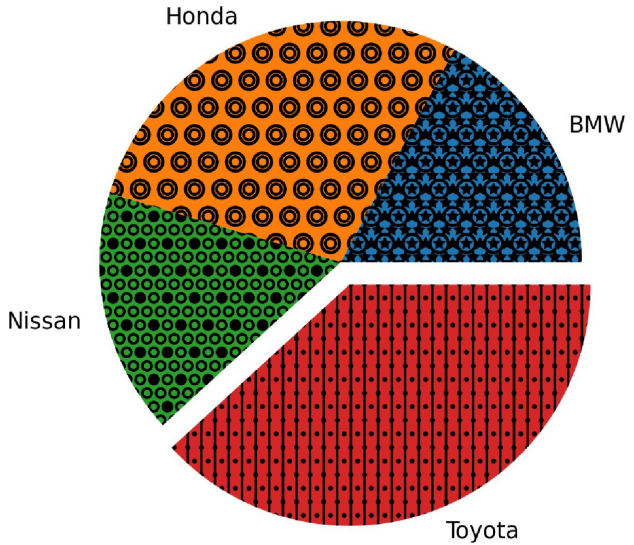
```
total
```

Out[6]:

Make	Price
BMW	2501927.0
Honda	4107844.0
Nissan	2391288.0
Toyota	5563201.0

In [15]:

```
explodes = [0, 0, 0, 0.1]
fig, ax = plt.subplots()
ax.pie(total["Price"], labels=total.index,
       explode=explodes,
       hatch=["**O", "oO", "O.O", ".||."])
plt.show()
```



Histogram

SECTION 03

The shape of one numeric column — how car prices are distributed.

Why a Histogram?

HISTOGRAM · THE QUESTION

One column: 1,000 prices

- A total or an average hides how the individual prices are spread.
- What is a typical price, and are most cars clustered around it?
- Are there a few unusually expensive cars pulling the average up?

- **Insight:** Bucket the prices into ranges and count each bucket — that count per range is a histogram.

In [16]:
```
data[["Make", "Price"]].head()
```

Out[16]:

	Make	Price
0	Honda	15323.0
1	BMW	19943.0
2	Honda	28343.0
3	Toyota	13434.0
4	Nissan	14043.0

Histogram with Matplotlib

Prepare the Data

HISTOGRAM · MATPLOTLIB

Check the column first

- One raw numeric column — no groupby needed.
- 950 prices, not 1,000 — 50 are missing.
- Min 2,796, median 14,297, max 52,458.

In [16]:

```
data[["Make", "Price"]].head()
```

Out[16]:

	Make	Price
0	Honda	15323.0
1	BMW	19943.0
2	Honda	28343.0
3	Toyota	13434.0
4	Nissan	14043.0

In [17]:

```
stats = data["Price"].describe()
stats[["count", "min", "50%", "max"]]
```

Out[17]:

	Price
count	950.0
min	2796.0
50%	14297.0
max	52458.0

- **Insight:** Both libraries skip the missing prices — but you should know they're there.

Histogram · Basic Plot

HISTOGRAM · MATPLOTLIB

One raw column in

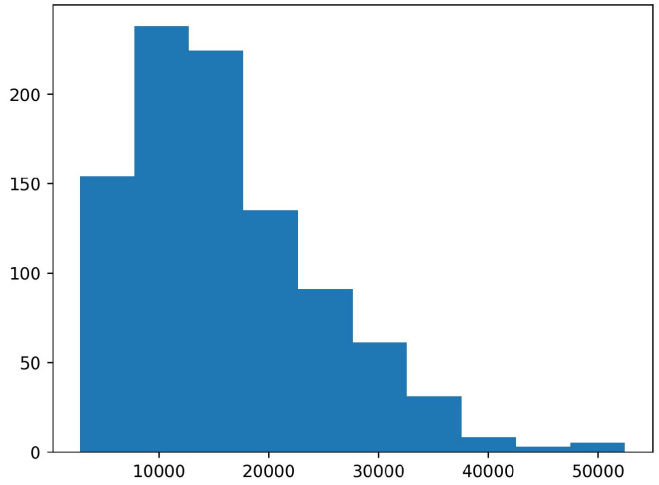
- plt.hist(column) buckets values into bins
- 10 bins by default and counts each bucket.
- No prep needed — the input on the left is the raw Price column.
- The right tail shows a few expensive outlier cars.

In [18]:
```
data[["Price"]].head(3)
```

Out[18]:

	Price
0	15323.0
1	19943.0
2	28343.0

In [19]:
```
plt.hist(data["Price"])
plt.show()
```



Bins & Axis Ranges

HISTOGRAM · MATPLOTLIB

Controlling the view

- bins=30 — more bins, finer detail.
- plt.xlim / plt.ylim clip the axes to the region you care about.
- Zooming to 0–40,000 drops the outlier tail from view.

In [18]:

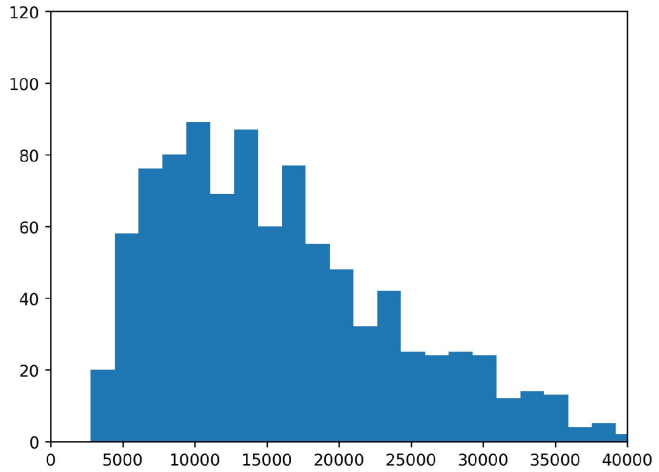
```
data[["Price"]].head(3)
```

Out[18]:

	Price
0	15323.0
1	19943.0
2	28343.0

In [20]:

```
plt.hist(data["Price"], bins=30)
plt.xlim(0, 40000)
plt.ylim(0, 120)
plt.show()
```



Histogram with Seaborn

Histogram · Seaborn

HISTOGRAM · SEABORN

Same idea, nicer defaults

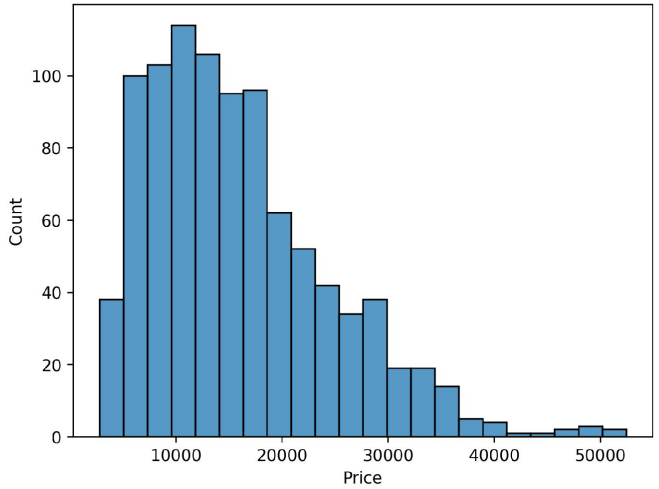
- sns.histplot(data=..., x=...) — DataFrame plus a column name, not an extracted array.
- Sensible bin count, labeled axes, styled bars — for free.

In [18]:
```
data[["Price"]].head(3)
```

Out[18]:

	Price
0	15323.0
1	19943.0
2	28343.0

In [21]:
```
sns.histplot(data=data, x="Price")
plt.show()
```



KDE + Groups

HISTOGRAM · SEABORN

Where Seaborn pulls ahead

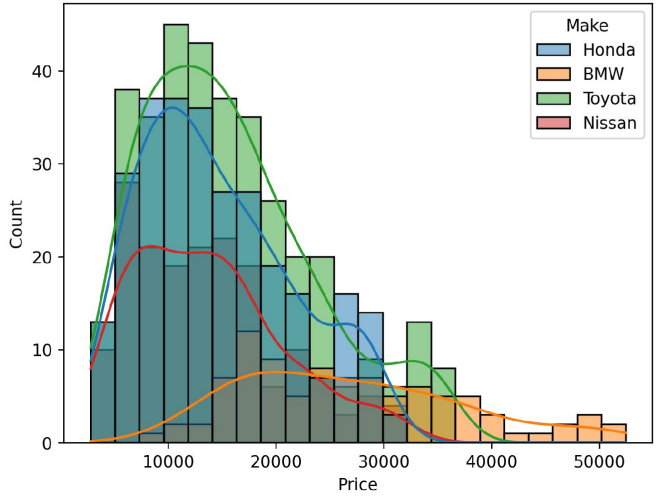
- kde=True overlays a smooth density curve on the bars.
- hue="Make" splits the histogram per maker — note the input now carries the Make column.
- Doing this in Matplotlib means loops and manual legends.

In [22]:
```
data[["Make", "Price"]].head(3)
```

Out[22]:

	Make	Price
0	Honda	15323.0
1	BMW	19943.0
2	Honda	28343.0

In [23]:
```
sns.histplot(data=data, x="Price",
             kde=True, hue="Make")
plt.show()
```



Scatter Plot

SECTION 04

Relationship between two numerics — does mileage push the price down?

Why a Scatter Plot?

SCATTER · THE QUESTION

Two numbers per car

- Buyers assume high mileage means a cheaper car — does the data agree?
- Does price fall as the odometer rises, and how strong is the link?
- Do different manufacturers follow the same pattern?

Insight: Plot one dot per car with mileage on x and price on y — the shape of the cloud is the answer.

In [24]:

```
data[["Odometer (KM)", "Price"]].head()
```

Out[24]:

	Odometer (KM)	Price
0	35431.0	15323.0
1	192714.0	19943.0
2	84714.0	28343.0
3	154365.0	13434.0
4	181577.0	14043.0

Scatterplot with Matplotlib

Scatter · Basic Plot

SCATTER · MATPLOTLIB

x column vs y column

- plt.scatter(x, y) — one dot per row of the table on the left.
- Clear downward trend: more kilometers, lower price.

In [26]:

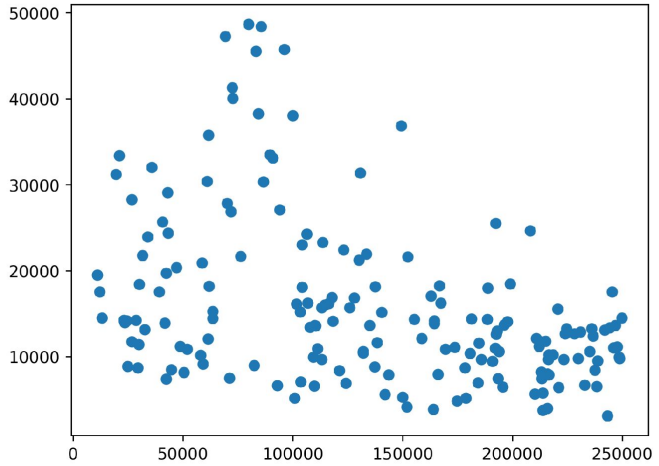
```
df.head(3)
```

Out[26]:

	Odometer (KM)	Price	Make
521	196130.0	13707.0	Nissan
737	223875.0	12650.0	Toyota
740	11049.0	19500.0	BMW

In [27]:

```
plt.scatter(df["Odometer (KM)"], df["Price"])
plt.show()
```



Color by Category

SCATTER · MATPLOTLIB

The manual way

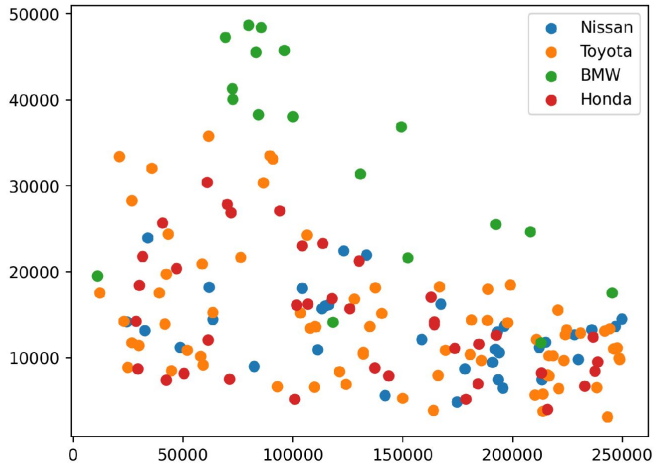
- Matplotlib has no hue concept like Seaborn — you loop over categories and call scatter once per maker.
- label= + plt.legend() assembles the legend by hand.
- Remember this loop: Seaborn deletes it next slide.

In [26]:
```
df.head(3)
```

Out[26]:

	Odometer (KM)	Price	Make
521	196130.0	13707.0	Nissan
737	223875.0	12650.0	Toyota
740	11049.0	19500.0	BMW

In [28]:
```
for make in df["Make"].dropna().unique():
    d = df[df["Make"] == make]
    plt.scatter(d["Odometer (KM)"], d["Price"],
                label=make)
plt.legend()
plt.show()
```



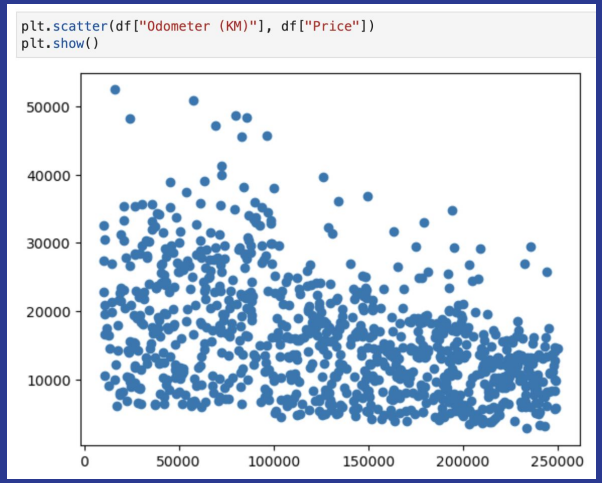
Sampling Dataset for smaller size

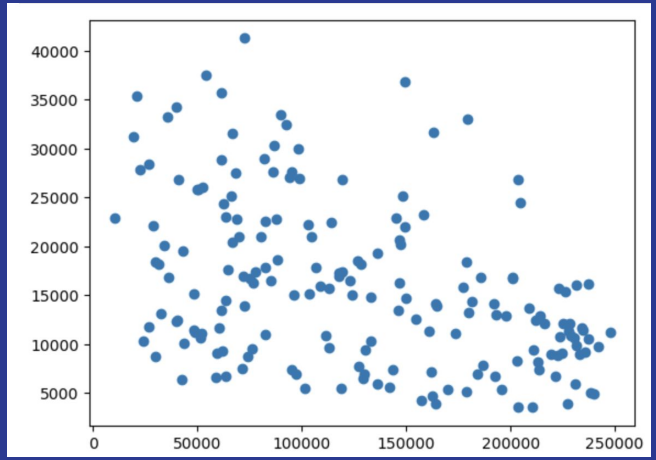
```
df = data.sample(frac=0.2)[["Odometer (KM)", "Price"]]
df
```

	Odometer (KM)	Price
605	42137.0	NaN
897	28809.0	22139.0
618	113538.0	NaN
961	239760.0	4896.0
765	108794.0	15962.0
...	...	...
342	97345.0	6962.0
318	94062.0	27057.0
437	43600.0	10030.0
746	201190.0	16705.0
993	162523.0	4696.0

200 rows × 2 columns

```
plt.scatter(df["Odometer (KM)"], df["Price"])
plt.show()
```





Scatterplot with Seaborn

Scatter · Seaborn

SCATTER · SEABORN

hue replaces the loop

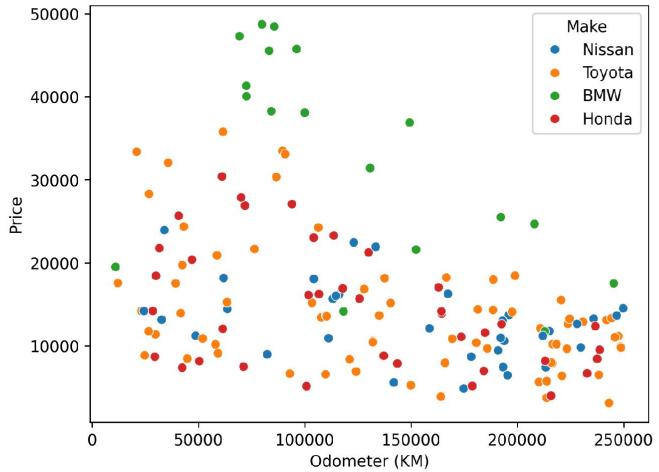
- hue="Make" colors the points per maker — one line instead of a for-loop.
- Legend, palette, axis labels: automatic.
- Same input table, a third of the code.

In [26]:
```
df.head(3)
```

Out[26]:

	Odometer (KM)	Price	Make
521	196130.0	13707.0	Nissan
737	223875.0	12650.0	Toyota
740	11049.0	19500.0	BMW

In [29]:
```
sns.scatterplot(data=df, x="Odometer (KM)",
                y="Price", hue="Make")
plt.show()
```



- **Insight:** hue also works on histplot, lineplot, boxplot… learn it once, use it everywhere.

Line Chart

SECTION 05

A value over time — how loan interest rates moved through 2018.

Why a Line Chart?

LINE CHART · THE QUESTION

49,523 loans, one per row

- The rows arrive in no particular order, so any trend over the year is hidden.
- Did interest rates rise or fall through 2018?
- Do 36-month and 60-month loans move together?

In [30]:
```
loans[["issue_d", "term", "int_rate"]].head()
```

Out[30]:

	issue_d	term	int_rate
0	2018-08-01	36 months	13.56
1	2018-03-01	36 months	10.90
2	2018-07-01	36 months	23.40
3	2018-12-01	36 months	14.47
4	2018-08-01	60 months	16.14

Insight: Average the rate per month, then connect the months in order — that is a line chart.

Prepare the Data

LINE CHART · MATPLOTLIB

Original → expected format

- pd.to_datetime turns "Aug-18" into real dates.
- groupby("issue_d").mean() — one rate per month.
- 49,523 rows in, 12 rows out.

In [2]:

```
loans = pd.read_csv("loans.csv")
loans[["loan_amnt", "term", "int_rate",
       "grade", "issue_d", "age"]].head()
```

Out[2]:

	loan_amnt	term	int_rate	grade	issue_d	age
0	5000	36 months	13.56	C	Aug-18	33
1	4000	36 months	10.90	B	Mar-18	21
2	5000	36 months	23.40	E	Jul-18	56
3	5500	36 months	14.47	C	Dec-18	20
4	11200	60 months	16.14	C	Aug-18	23

In [31]:

```
loans["issue_d"] = pd.to_datetime(
    loans["issue_d"], format="%b-%y")
monthly = loans.groupby("issue_d")["int_rate"] \
               .mean()
monthly.head()
```

Out[31]:

	int_rate
issue_d	
2018-01-01	13.542010
2018-02-01	13.536895
2018-03-01	13.518773
2018-04-01	13.440563
2018-05-01	13.434358

Line Chart with Matplotlib

Line · Basic Plot

LINE CHART · MATPLOTLIB

Markers, color, line style

- plt.plot(x, y) connects the monthly points on the left in order.
- marker="o" dots each data point;
- linestyle="--" dashes the line, other linestyle("-*", "-o");
- color= names any color.

In [32]:

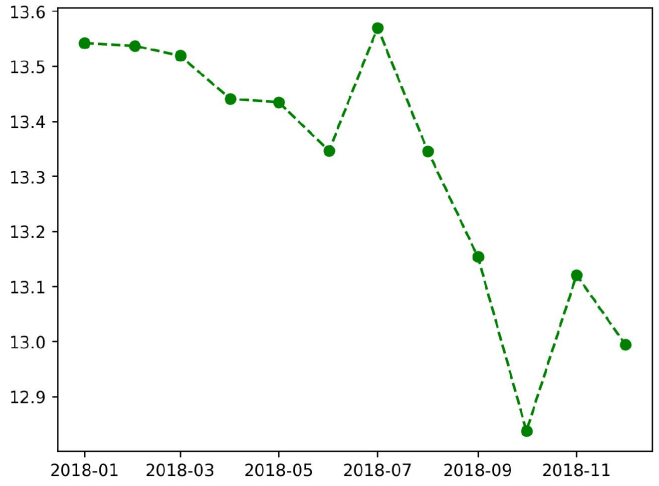
```
monthly.head(3)
```

Out[32]:

issue_d	int_rate
2018-01-01	13.542010
2018-02-01	13.536895
2018-03-01	13.518773

In [33]:

```
plt.plot(monthly.index, monthly.values,
         marker="o", color="green",
         linestyle="--")
plt.show()
```



Line · Multiple Series

LINE CHART · MATPLOTLIB

One plot call per line

- Filter, aggregate, and plot each term separately — 36 vs 60-month loans.
- plt.legend() collects the labels.
- Two prepared Series on the left → two lines on the right.

In [34]:

```
pd.DataFrame({"m36": m36.head(3),
              "m60": m60.head(3)})
```

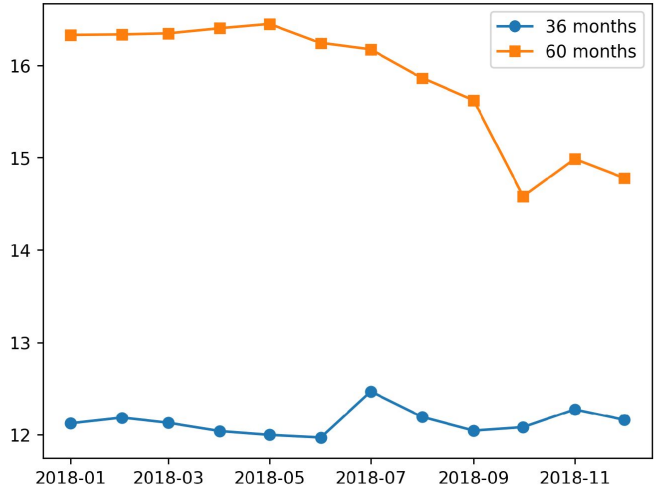
Out[34]:

issue_d	m36	m60
2018-01-01	12.123129	16.329229
2018-02-01	12.184810	16.334571
2018-03-01	12.129043	16.345960

In [35]:

```
m36 = loans[loans["term"] == " 36 months"] \
      .groupby("issue_d")["int_rate"].mean()
m60 = loans[loans["term"] == " 60 months"] \
      .groupby("issue_d")["int_rate"].mean()

plt.plot(m36.index, m36.values, marker="o",
         label="36 months")
plt.plot(m60.index, m60.values, marker="s",
         label="60 months")
plt.legend()
plt.show()
```



Line Chart with Seaborn

Line · Seaborn

LINE CHART · SEABORN

Raw 49,523 rows straight in

- No to_datetime groupby pipeline — sns.lineplot aggregates per month itself.
- hue="term" draws both loan terms with a legend.
- The shaded band is a 95% confidence interval — free statistics.

In [36]:
```
loans[["issue_d", "term", "int_rate"]].head(3)
```

Out[36]:

	issue_d	term	int_rate
0	2018-08-01	36 months	13.56
1	2018-03-01	36 months	10.90
2	2018-07-01	36 months	23.40

- **Insight:** The band needs the raw rows to compute spread — pre-aggregated data can't produce it.

In [37]:
```
sns.lineplot(data=loans, x="issue_d",
             y="int_rate", hue="term")
plt.show()
```

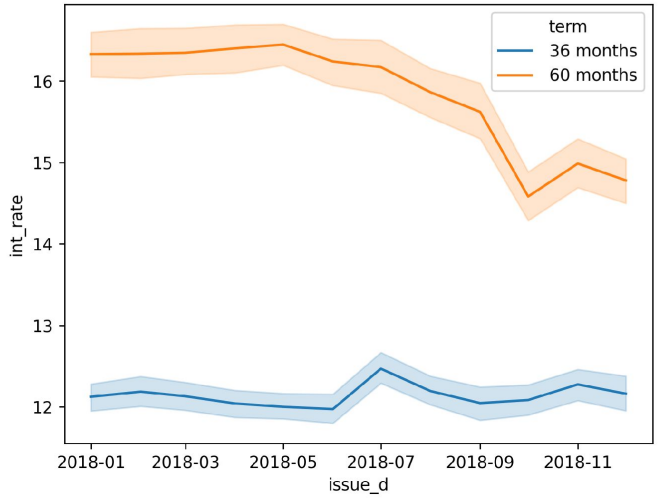


Chart Formatting

Formatting Any Chart

WORKS FOR BOTH LIBRARIES

Title, labels, size

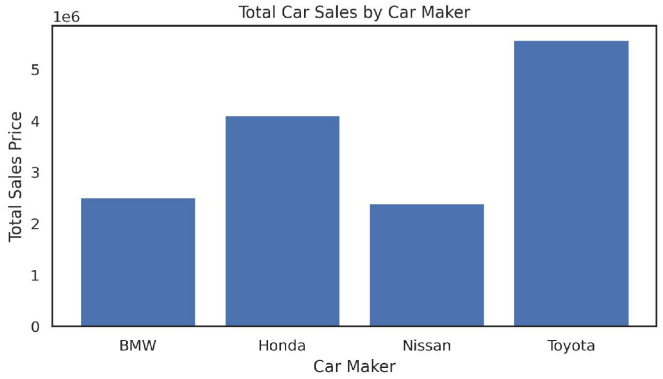
- figsize=(8, 4) sets width × height in inches.
- set_title / set_xlabel / set_ylabel — no chart ships without them.
- Seaborn charts are Matplotlib axes, so the same calls work on both.

In [6]:
```
total
```

Out[6]:

Make	Price
BMW	2501927.0
Honda	4107844.0
Nissan	2391288.0
Toyota	5563201.0

In [50]:
```
sns.set_theme(style="white")
fig, ax = plt.subplots(figsize=(8, 4))
ax.bar(total.index, total["Price"])
ax.set_title("Total Car Sales by Car Maker")
ax.set_xlabel("Car Maker")
ax.set_ylabel("Total Sales Price")
plt.show()
```



Subplots Dashboard

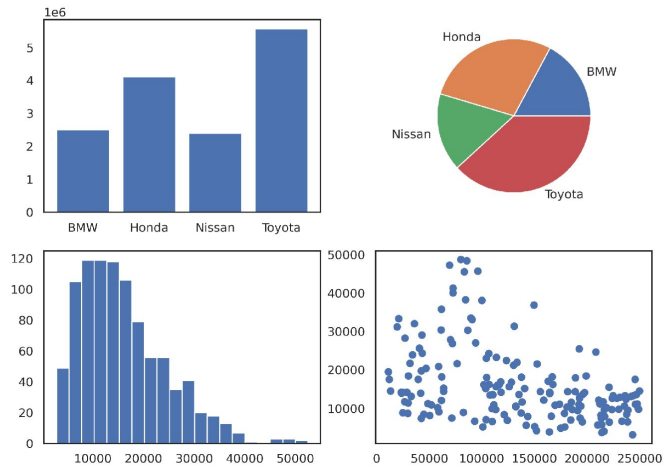
WORKS FOR BOTH LIBRARIES

One figure, four axes

- plt.subplots(2, 2) returns a grid of axes; index with ax[row, col].
- Each cell takes any chart from today — a one-figure dashboard.

In [51]:

```
fig, ax = plt.subplots(2, 2, figsize=(10, 7))
ax[0, 0].bar(total.index, total["Price"])
ax[0, 1].pie(total["Price"], labels=total.index)
ax[1, 0].hist(data["Price"], bins=20)
ax[1, 1].scatter(df["Odometer (KM)"], df["Price"])
plt.show()
```



Themes & Palettes

SEABORN EXTRAS

Restyle every chart at once

- sns.set_theme() changes the look globally — every later figure inherits it, Matplotlib ones included.
- style= controls the background grid; palette= the color cycle.
- Same data and same boxplot code — only the theme line changed.

In [42]:
```
loans[["grade", "int_rate"]].head(3)
```

Out[42]:

	grade	int_rate
0	C	13.56
1	B	10.90
2	E	23.40

In [49]:
```
sns.set_theme(style="darkgrid",
              palette="pastel")

sns.boxplot(data=loans, x="grade", y="int_rate",
            order=sorted(loans["grade"].unique()))
plt.show()
```

